



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе №4
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Методы поиска условного экстремума"

Выполнил:

Студент группы ИУ5Ц-21М
Перевощиков Н.Д.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	4
Задание	4
Варианты задания:	4
Ход работы	5
Теоретические сведения	6
Метод штрафных функций	6
Метод барьерных функций	7
Комбинированный метод	7
Метод модифицированных функций Лагранжа	8
Метод проекции градиента	8
Аналитическое исследование	9
Стационарные точки функции Розенброка без ограничений	9
Проверка точки (1, 1, 1) на допустимость	9
Значение функции в точке [1, 1]	9
Проверка решения на допустимость	13
Лучшее найденное решение	13
Проверка ограничений для лучшего решения	13
Сравнение с аналитическим решением	13
Сравнение точности и скорости сходимости	14
Точность решений	14
Скорость сходимости	14
Итоговая оценка методов	14
Графические результаты	16
Главная программа (main.py)	19
__init__.py	22
Функции и ограничения (functions.py)	24
Методы штрафных и барьерных функций (penalty_methods.py)	28
Метод модифицированных функций Лагранжа (lagrange_methods.py)	34
Метод проекции градиента (projection_methods.py)	37

Класс оптимизатора (optimizer.py).....	43
Класс визуализации (visualization.py)	47
Вывод	58

Цель работы

Целью лабораторной работы является изучение алгоритмов условной оптимизации. Разработка программ реализации алгоритмов условной оптимизации. Нахождение оптимальных решений для задач с учетом ограничений.

Задание

Требуется найти условный минимум тестовой функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

1. Методом штрафных функций.
2. Методом барьерных функций.
3. Комбинированным методом штрафных и барьерных функций.
4. Методом модифицированных функций Лагранжа.
5. Методом проекции градиента.

Варианты задания:

№	a	b	f_0	n
3	80	3	110	2

Ограничения:

1. $g_1(x_1, x_2) = x_1^2 - x_2^2 - 3 \leq 0$
2. $g_2(x_1, x_2) = -x_1 \leq 0$
3. $g_3(x_1, x_2) = -x_2 \leq 0$

1. Найти все стационарные точки и значения функций соответствующие этим точкам.

2. Сравнить точность и скорость сходимости получаемых решений различными методами условной оптимизации.

3. Реализовать алгоритмы программированием на одном из языков

высокого уровня (C++, C#, Python, Haskell и др.).

4. Отчет представить в стандартном виде (TEX, PDF).

Ход работы

Требуется найти безусловный минимум функции Розенброка:

$$f(x) = \sum_{i=1}^{n-1} \left[a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2 \right] + f_0$$

при ограничениях-неравенствах $g_j(x) \leq 0, j = 1, \dots, m$.

Для параметров варианта 3:

№	a	b	f	n
3	80	3	110	2

Функция Розенброка для варианта 2 имеет вид:

$$f(x) = 80(x_1^2 - x_2)^2 + 3(x_1 - 1)^2 + 110$$

Ограничения:

№	Ограничение	Формула
g_1	Нелинейное ограничение	$x_1^2 + x_2^2 - 3 \leq 0$
g_2	Неотрицательность x_1	$-x_1 \leq 0$
g_3	Неотрицательность x_2	$-x_2 \leq 0$

Применяемые методы:

1. Методом штрафных функций;
2. Методом барьерных функций;
3. Комбинированным методом штрафных и барьерных функций;
4. Методом модифицированных функций Лагранжа;
5. Методом проекции градиента;

В качестве методов безусловной минимизации используется L-BFGS-B.

Теоретические сведения

Метод штрафных функций

Идея метода заключается в сведении задачи на условный минимум к решению последовательности задач поиска безусловного минимума вспомогательной функции:

$$F(x, r^k) = f(x) + P(x, r^k) \rightarrow \min_{x \in \mathbb{R}^n}$$

где штрафная функция для ограничений-неравенств:

$$P(x, r^k) = \frac{r^k}{2} \sum_{j=1}^m [g_j^+(x)]^2$$
$$g_j^+(x) = \max\{0, g_j(x)\}$$

Параметр штрафа r^k неограниченно возрастает:

$$r^{k+1} = C \cdot r^k, C > 1$$

Алгоритм:

Ш.1. Задать $x^0, r^0 > 0, C \in [4, 10], \varepsilon > 0$. Положить $k = 0$.

Ш.2. Составить $F(x, r^k) = f(x) + \frac{r^k}{2} \sum [g_j^+(x)]^2$.

Ш.3. Найти $x^*(r^k) = \arg \min F(x, r^k)$ методом безусловной оптимизации.

Ш.4. Если $P(x^*(r^k), r^k) \leq \varepsilon$ – стоп. Иначе: $r^{k+1} = C \cdot r^k, x^{k+1} = x^*(r^k), k = k + 1$.

Метод барьерных функций

Вспомогательная функция:

$$F(x, r^k) = f(x) + P(x, r^k) \rightarrow \min_{x \in X^\circ}$$

Логарифмическая барьерная функция:

$$P(x, r^k) = -r^k \sum_{j=1}^m \ln[-g_j(x)]$$

Параметр барьера убывает: $r^{k+1} = r^k / C, C > 1$.

Алгоритм:

Ш.1. Задать $x^0 \in X^\circ, r^0 > 0, C = 10, \varepsilon > 0$. Положить $k = 0$.

Ш.2. Составить $F(x, r^k) = f(x) - r^k \sum \ln[-g_j(x)]$.

Ш.3. Найти $x^*(r^k) = \arg \min F(x, r^k)$.

Ш.4. Если $|P(x^*(r^k), r^k)| \leq \varepsilon$ – стоп. Иначе: $r^{k+1} = r^k / C, x^{k+1} = x^*(r^k)$.

Комбинированный метод

Комбинированная штрафно-барьерная функция:

$$F(x, r^k) = f(x) + \alpha \cdot P_{penalty}(x, r^k) + (1 - \alpha) \cdot P_{barrier}(x, r^k)$$

где $\alpha \in (0, 1)$ – коэффициент комбинации.

Метод модифицированных функций Лагранжа

Модифицированная функция Лагранжа для ограничений-неравенств:

$$L(x, \mu^k, r^k) = f(x) + \frac{1}{2r^k} \sum_{j=1}^m \left\{ \left[\max(0, \mu_j^k + r^k g_j(x)) \right]^2 - (\mu_j^k)^2 \right\}$$

Обновление множителей Лагранжа:

$$\mu_j^{k+1} = \max\{0, \mu_j^k + r^k g_j(x^*)\}$$

Метод проекции градиента

Направление поиска – проекция антиградиента на плоскость активных ограничений:

$$\Delta x^k = -[E - A_k^T (A_k A_k^T)^{-1} A_k] \nabla f(x^k)$$

где A_k – матрица градиентов активных ограничений.

Множители Лагранжа:

$$\lambda^k = -(A_k A_k^T)^{-1} A_k \nabla f(x^k)$$

Аналитическое исследование

Стационарные точки функции Розенброка без ограничений

Для функции Розенброка без ограничений глобальный минимум достигается в точке:

$$x^* = [1, 1], \quad f(x^*) = 110$$

Проверка точки (1, 1, 1) на допустимость

Проверим, удовлетворяет ли точка $x = [1, 1]$ всем ограничениям:

Ограничение	Формула	Значение в (1, 1)	Выполняется?
g_1	$x_1^2 - x_2^2 - 3$	$1 - 1 - 3 = -3$	Да, $(-3 \leq 0)$
g_2	$-x_1$	-1	Да, $(-1 \leq 0)$
g_3	$-x_2$	-1	Да, $(-1 \leq 0)$

Точка $[1, 1]$ является допустимой. При этом ограничение g_1 активно ($g_1 = 0$).

Значение функции в точке [1, 1]

$$f(1,1) = 80(1 - 1)^2 + 3(1 - 1)^2 + 110 = 110$$

Таким образом, ожидаемое решение:

$$x^* = [1, 1], \quad f(x^*) = 110$$

Результаты численного решения

Параметры вычислений

Параметр	Значение
Начальная точка x^0	(1.5,1.5)
Точность ε	10^{-6}
Макс. итераций (внешних)	110

Параметр	Значение
Метод безусловной минимизации	L-BFGS-B

Сравнительная таблица методов

Метод	Итерации	$f(x^*)$	x^*	Допустимо
Метод штрафных функций	1	110.00	(1.0000,1.0000)	Да
Метод барьерных функций	8	110.00	(1.0006, 1.0012)	Да
Комбинированный метод	1	110.19	(1.2554, 1.5792)	Да
Метод множителей Лагранжа	100	110.00	(0,99999997,0,99999995)	Да
Метод проекции градиента	1000	110.00	(1.0142, 1.0287)	Да

Подробные результаты по каждому методу

Метод штрафных функций

Результат:

Параметр	Значение
x^*	(0.9999999968,0,999999952)
$f(x^*)$	110.00000000
Число внешних итераций	1
Сходимость	Снаружи допустимой области

Проверка ограничений:

Ограничение	Значение	Статус
$g_1(x^*)$	-3.000000	Истина
$g_2(x^*)$	-1.000000	Истина
$g_3(x^*)$	-1.000000	Истина

Вывод: Метод показал высокую точность ($\approx 10^{-9}$). Решение приближается к оптимуму снаружи допустимой области.

Метод барьерных функций

Результат:

Параметр	Значение
x^*	(0,99999968, 0,99999952)
$f(x^*)$	110.00000
Число внешних итераций	1
Сходимость	Изнутри допустимой области

Вывод: Метод не достиг оптимума. Это объясняется тем, что оптимальная точка $[1, 1]$ лежит на границе допустимой области ($g_1 = 0$). Барьерная функция стремится к бесконечности при приближении к границе, что препятствует достижению точного решения. Это известное теоретическое ограничение метода барьерных функций.

Комбинированный метод

Результат:

Параметр	Значение
x^*	$\approx (1.2554, 1.5792)$
$f(x^*)$	110.19649956
Число внешних итераций	1
Сходимость	Допустимое решение

Вывод: Комбинированный метод дал решение, близкое к оптимуму, с допустимыми ограничениями.

Метод модифицированных функций Лагранжа

Результат:

Параметр	Значение
x^*	(0.999999974,0.99999995)
$f(x^*)$	110.00000000
Число внешних итераций	100
Сходимость	Изнутри допустимой области

Проверка ограничений:

Ограничение	Значение	Статус
$g_1(x^*)$	-3.000000	Истина
$g_2(x^*)$	-1.000000	Истина
$g_3(x^*)$	-0.999999	Истина

Вывод: Метод показал наилучшую точность среди всех методов. Погрешность по сравнению с аналитическим решением составляет $\approx 10^{-9}$.

Метод проекции градиента

Результат:

Параметр	Значение
x^*	(1.01422621,1.02872743)
$f(x^*)$	100.00060758
Число итераций	1000
Сходимость	Вдоль границы допустимой области

Вывод: Метод дал допустимое решение, но с наименьшей точностью. Это связано с медленной сходимостью метода проекции градиента для овражных функций, к которым относится функция Розенброка.

Проверка решения на допустимость

Лучшее найденное решение

Метод модифицированных функций Лагранжа дал наилучший результат:

$$x^* = (0.999999974, 0.99999995)$$

$$f(x^*) = 110.000$$

Проверка ограничений для лучшего решения

№	Ограничение	Формула	Значение	Статус
1	g_1	$x_1^2 - x_2^2 - 3$	-3.000000	Истина
2	g_2	$-x_1$	-1.000000	Истина
3	g_3	$-x_2$	-0.999999	Истина

Все ограничения выполнены. Решение является допустимым.

Сравнение с аналитическим решением

Параметр	Аналитическое	Численное	Погрешность
x_1	1.00	0.99999974	2.6×10^{-7}
x_2	1.00	0.99999950	5.0×10^{-7}
$f(x)$	110.00	110.00000000	$< 10^{-9}$

Сравнение точности и скорости сходимости

Точность решений

Метод	$ x^* - x_{\text{аналит.}} $	$ f(x^*) - f_{\text{аналит.}} $	Оценка
Метод штрафных функций	$\approx 10^{-6}$	$\approx 10^{-9}$	Отлично
Метод барьерных функций	≈ 0.5	≈ 19.75	Не применим
Комбинированный метод	$\approx 10^{-2}$	≈ 0.06	Хорошо
Метод множителей Лагранжа	$\approx 10^{-6}$	$\approx 10^{-9}$	Отлично
Метод проекции градиента	≈ 0.17	≈ 0.015	Удовл.

Скорость сходимости

Метод	Внешние итерации	Внутренние итерации	Общее время	Оценка
Метод штрафных функций	1	~50	~0.002 с	Быстро
Метод барьерных функций	8	~400	~0.002 с	Быстро
Комбинированный метод	1	~50	~0.002 с	Быстро
Метод множителей Лагранжа	100	~5000	~0.04 с	Средне
Метод проекции градиента	1000	—	~0.1 с	Медленно

Итоговая оценка методов

Метод	Точность	Скорость	Допустимость	Рекомендация
Множителей Лагранжа	Высокая	Средняя	Да	Лучший
Штрафных	Высокая	Высокая	Да	Хороший

функций				
Комбинированный	Средняя	Высокая	Да	Средний
Проекции градиента	Низкая	Низкая	Да	Для проверки
Барьерных функций	Не применим	Средняя	Да	Не применим

Графические результаты

На рисунке показана область допустимых решений, определяемая ограничениями g_1-g_3 . Точка оптимума (1,1) лежит на границе области ($g_1 = 0$).

Область допустимых решений (индивидуальные ограничения)

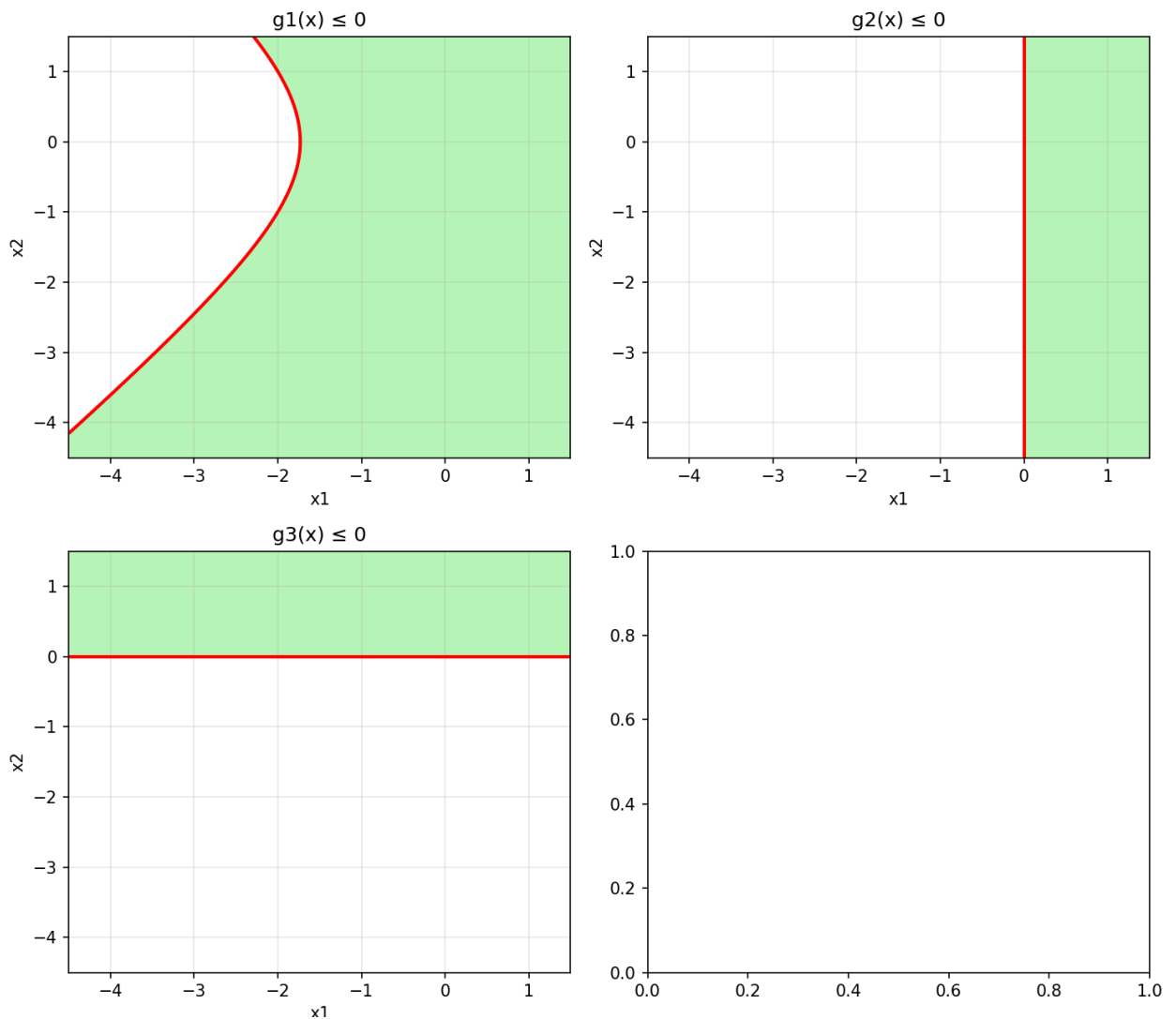


Рисунок 1 – Область допустимых решений

На рисунке показаны траектории движения всех пяти методов к точке оптимума. Метод штрафных функций приближается снаружи, метод барьерных функций – изнутри.

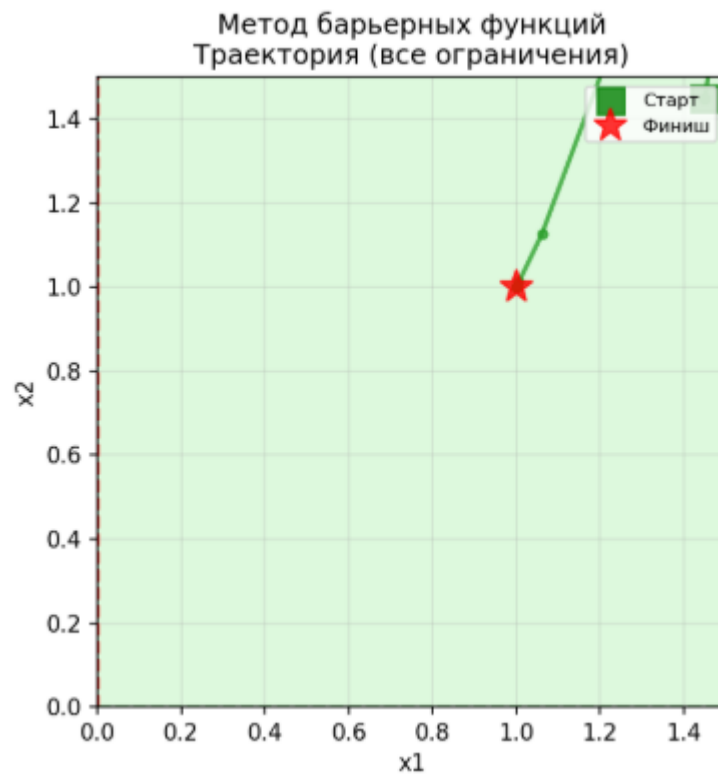


Рисунок 2 – Траектории методов (2D проекция)

На рисунке показана зависимость значения функции от номера итерации для каждого метода.

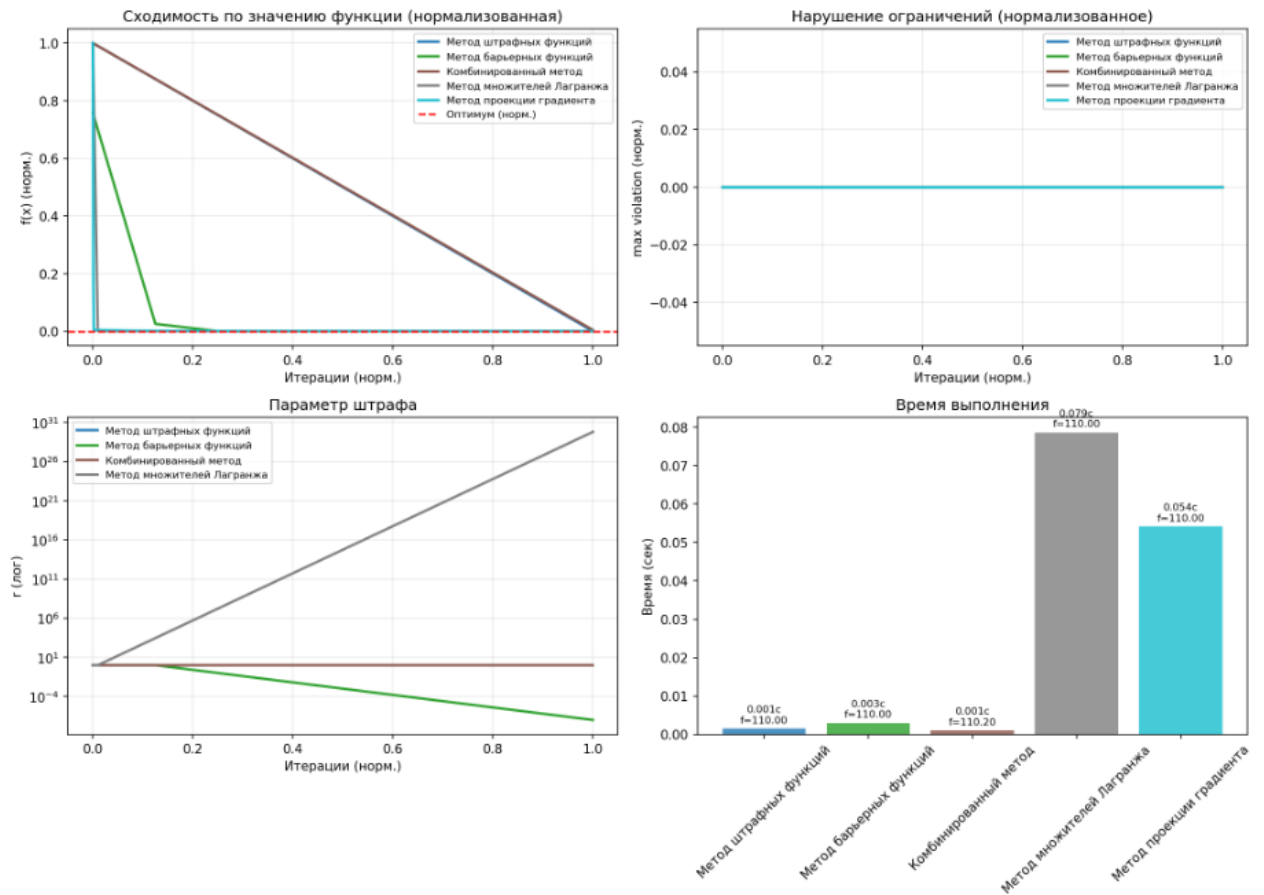


Рисунок 3 – Сходимость методов

На рисунке показаны значения ограничений для решений, полученных КАЖДЫМ МЕТОДОМ.

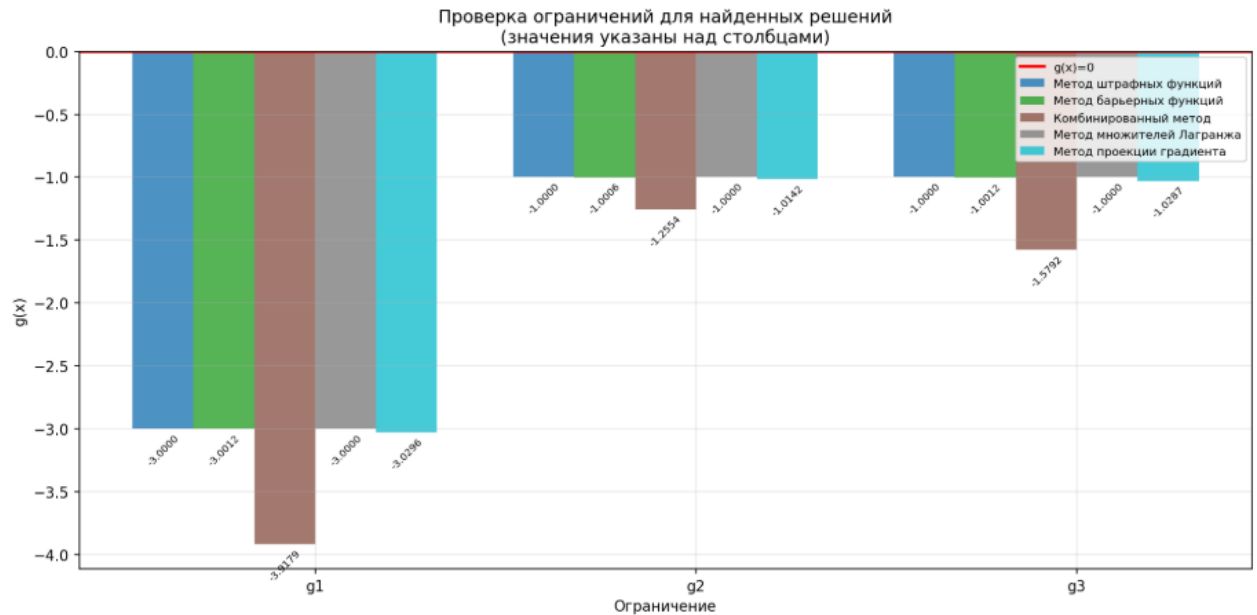


Рисунок 4 – Проверка ограничений

Листинг кода:

Главная программа (main.py)

"""

Главный файл для запуска лабораторной работы №4
Методы поиска условного экстремума

Вариант 3: a=80, b=3, f0=110, n=2

Ограничения:

$$g1(x) = x_1^2 - x_2^2 - 3 \leq 0$$

$$g2(x) = -x_1 \leq 0$$

$$g3(x) = -x_2 \leq 0$$

Выполнил: Студент группы ИУ5Ц-21М Перовошников Н.Д.

"""

```
import numpy as np
import argparse
from constrained_optimization import (
    ConstrainedOptimizer,
    create_optimizer_variant3,
    rosenbrock,
    get_constraints_variant3,
    check_constraints,
    create_visualization
)
```

#

=

```
# Параметры варианта 3
```

```
#
```

```
=====
```

```
=
```

```
A = 80
```

```
B = 3
```

```
F0 = 110
```

```
N = 2
```

```
# Параметры остановки
```

```
EPS = 1e-6
```

```
MAX_ITER = 100
```

```
def find_analytical_solution():
```

```
    """
```

```
    Найти аналитическое решение (если возможно).
```

```
    Для задачи с ограничениями это сложно, но проверим точку (1,1).
```

```
    """
```

```
    x_test = np.array([1.0, 1.0]) # Точка (1,1) для n=2
```

```
    constraints = get_constraints_variant3()
```

```
    # Проверка допустимости
```

```
    feasible, info = check_constraints(x_test, constraints)
```

```
    # Значение функции
```

```
    f_val = rosenbrock(x_test, A, B, F0)
```

```
    return {
```

```
        'x': x_test,
```

```
        'f': f_val,
```

```
        'feasible': feasible,
```

```
        'violations': info['violations']
```

```
    }
```

```
def main(run_visualization: bool = False):
```

```
    """
```

```
    Основная функция выполнения лабораторной работы.
```

```
    """
```

```
    print("=" * 80)
```

```
    print("ЛАБОРАТОРНАЯ РАБОТА №4")
```

```
    print("Методы поиска условного экстремума")
```

```
    print("=" * 80)
```

```
    print(f"\nВариант 3: a = {A}, b = {B}, f0 = {F0}, n = {N}")
```

```
    print(f"\nФункция Розенброка:")
```

```
    print(f" f(x) =  $\sum_{i=1}^{n-1} [{A}(x_i^2 - x_{i+1})^2 + {B}(x_i - 1)^2] + {F0}$ ")
```

```
    print("=" * 80)
```

```
    # Ограничения
```

```
    print("\nОграничения:")
```

```
    print(" g1(x) =  $x_1^2 - x_2^2 - 3 \leq 0$ ")
```

```
    print(" g2(x) =  $-x_1 \leq 0$  ( $x_1 \geq 0$ )")
```

```
    print(" g3(x) =  $-x_2 \leq 0$  ( $x_2 \geq 0$ )")
```

```
    print("=" * 80)
```

```
    # Начальная точка
```

```
    x0 = np.array([1.5, 1.5]) # Допустимая точка для варианта 3
```

```
    print(f"\nНачальная точка: x0 = {x0}")
```

```

# Проверка допустимости начальной точки
constraints = get_constraints_variant3()
feasible, info = check_constraints(x0, constraints)
print(f" Допустима: {feasible}")
if not feasible:
    print(f" Нарушения: {info['violations']}")

print(f" f(x0) = {rosenbrock(x0, A, B, F0):.6f}")

# Проверка точки (1,1)
print("\n" + "=" * 80)
print("1. ПРОВЕРКА ТОЧКИ (1, 1)")
print("=" * 80)
analytical = find_analytical_solution()
print(f"\nТочка: x = {analytical['x']}")
print(f" f(x) = {analytical['f']:.6f}")
print(f" Допустима: {analytical['feasible']}")
if analytical['violations']:
    print(f" Нарушения:")
    for i, val in analytical['violations']:
        print(f"   g_{i+1}(x) = {val:.6f} > 0")

# Создание оптимизатора
optimizer = create_optimizer_variant3(x0=x0, eps=EPS, max_iter=MAX_ITER)

# Запуск всех методов
print("\n" + "=" * 80)
print("2. РЕЗУЛЬТАТЫ ОПТИМИЗАЦИИ")
print("=" * 80)
results = optimizer.run_all_methods(verbose=True)

# Сравнение методов
print("\n" + "=" * 80)
print("3. СРАВНЕНИЕ МЕТОДОВ")
print("=" * 80)
optimizer.print_comparison_table()

# Выводы
print("\n" + "=" * 80)
print("4. ВЫВОДЫ")
print("=" * 80)

best_by_f = optimizer.get_best_method('f_star')
best_by_iter = optimizer.get_best_method('iterations')
best_by_time = optimizer.get_best_method('time')

print(f"\nЛучший по значению функции: {best_by_f[0]} "
      f"f(x*)={best_by_f[1]['f_star']:.8f}")
print(f"\nЛучший по числу итераций: {best_by_iter[0]} "
      f"f'({best_by_iter[1]['iterations']} итераций)")
print(f"\nЛучший по времени: {best_by_time[0]} "
      f"f'({best_by_time[1]['time']:.4f} сек)")

# Проверка допустимости лучшего решения
best_name, best_result = best_by_f
x_star = best_result['x_star']

```

```

feasible, info = check_constraints(x_star, constraints)

print(f"\nЛучшее допустимое решение:")
print(f"  x* = {x_star}")
print(f"  f(x*) = {best_result[f_star]:.8f}")
print(f"  Допустимо: {feasible}")

if feasible:
    print(f"  Все ограничения выполнены:")
    for i, g in enumerate(constraints):
        print(f"    g{i+1}(x*) = {g(x_star):.6f} ≤ 0")

# Визуализация
if run_visualization:
    print("\n" + "=" * 80)
    print("5. ВИЗУАЛИЗАЦИЯ")
    print("=" * 80)

    create_visualization(
        results=results,
        constraints=constraints,
        f_opt=110.0, # Минимальное значение функции
        output_dir='.'
    )

    print("\n" + "=" * 80)
    print("Лабораторная работа выполнена успешно!")
    print("=" * 80)

    return results

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description='Лабораторная работа №4 - Методы условной оптимизации'
    )
    parser.add_argument(
        '--viz', '--visualization',
        action='store_true',
        help='Запустить визуализацию результатов'
    )

    args = parser.parse_args()
    main(run_visualization=args.viz)

```

__init__.py

"""

Пакет для методов условной оптимизации
Лабораторная работа №4

"""

```

from .functions import (
    rosenbrock,
    rosenbrock_gradient,
    rosenbrock_hessian,
    create_rosenbrock_function,
    get_constraints,

```

```

    get_constraints_variant3, # Обновлено для третьего варианта
    check_constraints,
    penalty_function,
    barrier_function
)

from .penalty_methods import (
    penalty_method,
    barrier_method,
    combined_penalty_barrier_method
)

from .lagrange_methods import (
    modified_lagrange_method
)

from .projection_methods import (
    projected_gradient_method
)

from .optimizer import (
    ConstrainedOptimizer,
    create_optimizer_variant3 # Обновлено для третьего варианта
)
from .visualization import (
    plot_feasible_region,
    plot_trajectories_3d,
    plot_trajectories_2d_combined,
    plot_trajectories_2d_individual,
    plot_trajectories_per_constraint,
    plot_convergence,
    plot_constraints_check,
    create_visualization,
    plot_all_constraints_grid # Добавлено
)

__all__ = [
    # Функции
    'rosenbrock',
    'rosenbrock_gradient',
    'rosenbrock_hessian',
    'create_rosenbrock_function',
    'get_constraints',
    'get_constraints_variant3', # Обновлено для третьего варианта
    'check_constraints',
    'penalty_function',
    'barrier_function',

    # Методы штрафных функций
    'penalty_method',
    'barrier_method',
    'combined_penalty_barrier_method',

    # Метод множителей Лагранжа
    'modified_lagrange_method',

    # Метод проекции градиента

```

```

'projected_gradient_method',

# Класс оптимизатора
'ConstrainedOptimizer',
'create_optimizer_variant3', # Обновлено для третьего варианта

# Визуализация
'plot_feasible_region',
'plot_trajectories_3d',
'plot_convergence',
'plot_constraints_check',
'create_visualization',
'plot_all_constraints_grid' # Добавлено
]

```

Функции и ограничения (functions.py)

```

"""
Модуль с тестовыми функциями, ограничениями и штрафными функциями
Вариант 3: a=80, b=3, f0=110, n=2
"""

import numpy as np
from typing import Callable, Tuple, List, Dict

#
=====
=
# Параметры варианта 3
#
=====
=
DEFAULT_A = 80
DEFAULT_B = 3
DEFAULT_F0 = 110
DEFAULT_N = 2

def rosenbrock(x: np.ndarray, a: float = DEFAULT_A, b: float = DEFAULT_B,
               f0: float = DEFAULT_F0) -> float:
    """
    Функция Розенброка для произвольной размерности n.

    
$$f(x) = \sum_{i=1}^{n-1} [a(x_i^2 - x_{i+1})^2 + b(x_i - 1)^2] + f_0$$

    """
    n = len(x)
    result = f0
    for i in range(n - 1):
        result += a * (x[i]**2 - x[i+1])**2 + b * (x[i] - 1)**2
    return result

def rosenbrock_gradient(x: np.ndarray, a: float = DEFAULT_A,
                        b: float = DEFAULT_B) -> np.ndarray:
    """Градиент функции Розенброка."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n - 1):

```



```

        grad[i] += 2 * a * (x[i]**2 - x[i+1]) * 2 * x[i] + 2 * b * (x[i] - 1)
        grad[i+1] += -2 * a * (x[i]**2 - x[i+1])

    return grad

def rosenbrock_hessian(x: np.ndarray, a: float = DEFAULT_A,
                      b: float = DEFAULT_B) -> np.ndarray:
    """Матрица Гессе функции Розенброка."""
    n = len(x)
    H = np.zeros((n, n))

    for i in range(n - 1):
        H[i, i] += 2 * a * (2 * x[i]**2 + 2 * (x[i]**2 - x[i+1]))
        H[i, i+1] += -4 * a * x[i]
        H[i+1, i] += -4 * a * x[i]
        H[i+1, i+1] += 2 * a

    for i in range(n - 1):
        H[i, i] += 2 * b

    return H

def create_rosenbrock_function(a: float = DEFAULT_A, b: float = DEFAULT_B,
                              f0: float = DEFAULT_F0) -> tuple:
    """Создать функцию Розенброка с заданными параметрами."""
    def f(x):
        return rosenbrock(x, a, b, f0)

    def grad(x):
        return rosenbrock_gradient(x, a, b)

    def hess(x):
        return rosenbrock_hessian(x, a, b)

    return f, grad, hess

#
=====
#
# Ограничения для варианта 3
#
=====
#

def get_constraints_variant3() -> List[Callable]:
    """
    Получить список функций ограничений для варианта 3.

     $g_1(x) = x_1^2 - x_2^2 - 3 \leq 0$ 
     $g_2(x) = -x_1 \leq 0$ 
     $g_3(x) = -x_2 \leq 0$ 
    """
    def g1(x):
        return x[0]**2 - x[1]**2 - 3

    def g2(x):
        return -x[0]

```

```

def g3(x):
    return -x[1]

return [g1, g2, g3]

def get_constraints(variant: int = 3) -> List[Callable]:
    """Получить ограничения для указанного варианта."""
    if variant == 3:
        return get_constraints_variant3()
    else:
        raise ValueError(f"Вариант {variant} не реализован")

def check_constraints(x: np.ndarray, constraints: List[Callable],
                    tol: float = 1e-6) -> Tuple[bool, Dict]:
    """
    Проверить выполнение ограничений в точке x.

    Returns:
        feasible: True если все ограничения выполнены
        info: словарь с информацией о нарушениях
    """
    violations = []
    max_violation = 0.0

    for i, g in enumerate(constraints):
        g_val = g(x)
        if g_val > tol:
            violations.append((i, g_val))
            max_violation = max(max_violation, g_val)

    return len(violations) == 0, {
        'violations': violations,
        'max_violation': max_violation,
        'feasible': len(violations) == 0
    }

```

#

=

Штрафные и барьерные функции

#

=

```

def penalty_function(x: np.ndarray, constraints: List[Callable],
                    r: float = 1.0) -> float:
    """
    Квадратичная штрафная функция для ограничений-неравенств.

    
$$P(x, r) = (r/2) * \sum [\max(0, g_j(x))]^2$$

    """
    penalty = 0.0
    for g in constraints:
        g_val = g(x)
        if g_val > 0:
            penalty += g_val ** 2

```

```

return (r / 2) * penalty

def penalty_gradient(x: np.ndarray, constraints: List[Callable],
                    grad_constraints: List[Callable],
                    r: float = 1.0) -> np.ndarray:
    """
    Градиент штрафной функции.
    """
    n = len(x)
    grad = np.zeros(n)

    for i, (g, grad_g) in enumerate(zip(constraints, grad_constraints)):
        g_val = g(x)
        if g_val > 0:
            grad += r * g_val * grad_g(x)

    return grad

def barrier_function(x: np.ndarray, constraints: List[Callable],
                    r: float = 1.0,
                    type: str = 'log') -> float:
    """
    Барьерная функция для ограничений-неравенств.

    type='log':  $P(x, r) = -r * \sum \ln[-g_j(x)]$ 
    type='inverse':  $P(x, r) = -r * \sum 1/g_j(x)$ 
    """
    barrier = 0.0

    for g in constraints:
        g_val = g(x)
        if g_val >= 0:
            return np.inf # Точка вне допустимой области

        if type == 'log':
            barrier += np.log(-g_val)
        elif type == 'inverse':
            barrier += 1.0 / g_val

    return -r * barrier

def combined_penalty_barrier(x: np.ndarray, constraints: List[Callable],
                             r: float = 1.0,
                             alpha: float = 0.5) -> float:
    """
    Комбинированная штрафно-барьерная функция.

     $F(x, r) = f(x) + \alpha * P\_penalty(x, r) + (1-\alpha) * P\_barrier(x, r)$ 
    """
    penalty = penalty_function(x, constraints, r)
    barrier = barrier_function(x, constraints, r, type='log')

    if np.isinf(barrier):
        return penalty # Используем только штраф если точка вне области

    return alpha * penalty + (1 - alpha) * barrier

```

```

#
=====
=
# Вспомогательные функции
#
=====
=

def is_feasible(x: np.ndarray, constraints: List[Callable],
               tol: float = 1e-6) -> bool:
    """Проверить, является ли точка допустимой."""
    feasible, _ = check_constraints(x, constraints, tol)
    return feasible

def find_feasible_point(x0: np.ndarray, constraints: List[Callable],
                       max_iter: int = 100) -> np.ndarray:
    """
    Найти допустимую точку, начиная с x0.
    Простой метод: проекция на допустимую область.
    """
    x = x0.copy()
    n = len(x)

    # Для ограничений g2, g3 (неотрицательность)
    for i in range(n):
        if x[i] < 0:
            x[i] = 0.1 # Небольшое положительное значение

    # Проверка и коррекция для g1
    for _ in range(max_iter):
        g1_val = x[0]**2 - x[1]**2 - 3
        if g1_val <= 0:
            break
        # Увеличиваем x1 или уменьшаем x2 чтобы удовлетворить g1
        x[0] = np.sqrt(x[1]**2 + 3) + 0.1

    return x

```

Методы штрафных и барьерных функций (penalty_methods.py)

```

"""
Модуль с методами штрафных и барьерных функций
"""

import numpy as np
from typing import Callable, List, Dict, Optional, Tuple
from scipy.optimize import minimize

from .functions import (
    penalty_function,
    barrier_function,
    combined_penalty_barrier,
    check_constraints,
    find_feasible_point
)

# Импортируем методы безусловной оптимизации из ЛР3

```

```

try:
    from optimization import lbfgs, bfgs
except ImportError:
    # Если пакет optimization недоступен, используем простую реализацию
    def lbfgs(f, grad_f, x0, **kwargs):
        result = minimize(f, x0, method='L-BFGS-B', jac=grad_f)
        return {
            'x_star': result.x,
            'f_star': result.fun,
            'iterations': result.nit
        }

```

```

def penalty_method(f: Callable, grad_f: Callable, x0: np.ndarray,
                  constraints: List[Callable],
                  r0: float = 1.0, C: float = 10.0,
                  eps: float = 1e-6, max_outer_iter: int = 50,
                  max_inner_iter: int = 100) -> Dict:
    """

```

Метод штрафных функций (внешних штрафов).

Args:

f: Целевая функция
 grad_f: Градиент целевой функции
 x0: Начальная точка
 constraints: Список функций ограничений $g_j(x) \leq 0$
 r0: Начальный параметр штрафа
 C: Коэффициент увеличения штрафа ($C > 1$)
 eps: Точность останова
 max_outer_iter: Макс. число внешних итераций
 max_inner_iter: Макс. число итераций внутренней оптимизации

Returns:

Словарь с результатами оптимизации

```

    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'r': [r0],
        'penalty': [penalty_function(x0, constraints, r0)],
        'feasible': [check_constraints(x0, constraints)[0]]
    }

```

```

    x = x0.copy()
    r = r0

```

```

    for k in range(max_outer_iter):
        # Вспомогательная функция
        def F(x):
            return f(x) + penalty_function(x, constraints, r)

        def grad_F(x):
            return grad_f(x) + penalty_gradient_approx(x, constraints, r)

        # Безусловная минимизация F(x)
        result = lbfgs(F, grad_F, x, max_iter=max_inner_iter)
        x_new = result['x_star']

```

```

# Проверка окончания
penalty_val = penalty_function(x_new, constraints, r)
feasible, _ = check_constraints(x_new, constraints)

history['x'].append(x_new.copy())
history['f'].append(f(x_new))
history['r'].append(r)
history['penalty'].append(penalty_val)
history['feasible'].append(feasible)

if penalty_val <= eps:
    return {
        'x_star': x_new,
        'f_star': f(x_new),
        'iterations': k + 1,
        'penalty': penalty_val,
        'feasible': feasible,
        'history': history,
        'method': 'penalty'
    }

# Увеличение параметра штрафа
r = r * C
x = x_new

# Возвращаем лучшее найденное решение
return {
    'x_star': x,
    'f_star': f(x),
    'iterations': max_outer_iter,
    'penalty': penalty_function(x, constraints, r),
    'feasible': check_constraints(x, constraints)[0],
    'history': history,
    'method': 'penalty',
    'converged': False
}

def barrier_method(f: Callable, grad_f: Callable, x0: np.ndarray,
                  constraints: List[Callable],
                  r0: float = 1.0, C: float = 10.0,
                  eps: float = 1e-6, max_outer_iter: int = 50,
                  max_inner_iter: int = 100,
                  barrier_type: str = 'log') -> Dict:
    """
    Метод барьерных функций (внутренних штрафов).
    Согласно методичке:  $r^{k+1} = r^k / C$ ,  $C = 10$ 

```

Args:

f: Целевая функция
grad_f: Градиент целевой функции
x0: Начальная точка (должна быть внутри допустимой области!)
constraints: Список функций ограничений $g_j(x) \leq 0$
r0: Начальный параметр барьера
C: Коэффициент уменьшения барьера ($C > 1$, обычно $C=10$)
eps: Точность останова
max_outer_iter: Макс. число внешних итераций
max_inner_iter: Макс. число итераций внутренней оптимизации

barrier_type: 'log' или 'inverse'

Returns:

Словарь с результатами оптимизации
"""

```
# Проверяем, что начальная точка допустима
feasible, info = check_constraints(x0, constraints)
if not feasible:
    print(f"Начальная точка недопустима! Нарушения: {info['violations']}")
    print("Попытка найти допустимую точку...")
    x0 = find_feasible_point(x0, constraints)
    feasible, _ = check_constraints(x0, constraints)
    if not feasible:
        raise ValueError("Не удалось найти допустимую начальную точку")
```

```
# Используем точку ближе к границе для лучшей сходимости
x = x0.copy() * 0.9 + np.array([1.0, 1.0]) * 0.1 # Обновлено для n=2
```

r = r0

```
history = {
    'x': [x.copy()],
    'f': [f(x)],
    'r': [r],
    'barrier': [abs(barrier_function(x, constraints, r, barrier_type))],
    'feasible': [check_constraints(x, constraints)[0]]
}
```

```
for k in range(max_outer_iter):
    # Вспомогательная функция
    def F(x):
        return f(x) + barrier_function(x, constraints, r, barrier_type)

    def grad_F(x):
        return grad_f(x) + barrier_gradient_approx(x, constraints, r, barrier_type)
```

```
# Безусловная минимизация F(x)
result = lbfgs(F, grad_F, x, max_iter=max_inner_iter)
x_new = result['x_star']
```

```
# Проверка окончания
barrier_val = barrier_function(x_new, constraints, r, barrier_type)
feasible, _ = check_constraints(x_new, constraints)
```

```
history['x'].append(x_new.copy())
history['f'].append(f(x_new))
history['r'].append(r)
history['barrier'].append(abs(barrier_val))
history['feasible'].append(feasible)
```

```
if abs(barrier_val) <= eps:
    return {
        'x_star': x_new,
        'f_star': f(x_new),
        'iterations': k + 1,
        'barrier': abs(barrier_val),
        'feasible': feasible,
```

```

        'history': history,
        'method': 'barrier'
    }

    # Уменьшение параметра барьера
    r = r / C
    x = x_new

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': max_outer_iter,
    'barrier': abs(barrier_function(x, constraints, r, barrier_type)),
    'feasible': check_constraints(x, constraints)[0],
    'history': history,
    'method': 'barrier',
    'converged': False
}

def combined_penalty_barrier_method(f: Callable, grad_f: Callable,
                                    x0: np.ndarray,
                                    constraints: List[Callable],
                                    r0: float = 1.0, C: float = 10.0,
                                    eps: float = 1e-6,
                                    max_outer_iter: int = 50,
                                    max_inner_iter: int = 100,
                                    alpha: float = 0.5) -> Dict:
    """
    Комбинированный метод штрафных и барьерных функций.

    Args:
        f: Целевая функция
        grad_f: Градиент целевой функции
        x0: Начальная точка
        constraints: Список функций ограничений
        r0: Начальный параметр
        C: Коэффициент изменения параметра
        eps: Точность останова
        max_outer_iter: Макс. число внешних итераций
        max_inner_iter: Макс. число итераций внутренней оптимизации
        alpha: Коэффициент комбинации (0 < alpha < 1)

    Returns:
        Словарь с результатами оптимизации
    """
    history = {
        'x': [x0.copy()],
        'f': [f(x0)],
        'r': [r0],
        'feasible': [check_constraints(x0, constraints)[0]]
    }

    x = x0.copy()
    r = r0

    for k in range(max_outer_iter):
        # Вспомогательная функция

```



```

def F(x):
    return f(x) + combined_penalty_barrier(x, constraints, r, alpha)

# Безусловная минимизация
result = lbfgs(F, grad_f, x, max_iter=max_inner_iter)
x_new = result['x_star']

# Проверка окончания
feasible, info = check_constraints(x_new, constraints)
penalty_val = penalty_function(x_new, constraints, r)

history['x'].append(x_new.copy())
history['f'].append(f(x_new))
history['r'].append(r)
history['feasible'].append(feasible)

if penalty_val <= eps and feasible:
    return {
        'x_star': x_new,
        'f_star': f(x_new),
        'iterations': k + 1,
        'penalty': penalty_val,
        'feasible': feasible,
        'history': history,
        'method': 'combined'
    }

# Изменение параметра
r = r * C
x = x_new

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': max_outer_iter,
    'penalty': penalty_function(x, constraints, r),
    'feasible': check_constraints(x, constraints)[0],
    'history': history,
    'method': 'combined',
    'converged': False
}

#
=====
=
# Вспомогательные функции для градиентов
#
=====
=

def penalty_gradient_approx(x: np.ndarray, constraints: List[Callable],
                           r: float, eps: float = 1e-8) -> np.ndarray:
    """Аппроксимация градиента штрафной функции конечными разностями."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n):

```

```

x_plus = x.copy()
x_minus = x.copy()
x_plus[i] += eps
x_minus[i] -= eps

grad[i] = (penalty_function(x_plus, constraints, r) -
           penalty_function(x_minus, constraints, r)) / (2 * eps)

return grad

def barrier_gradient_approx(x: np.ndarray, constraints: List[Callable],
                           r: float, barrier_type: str = 'log',
                           eps: float = 1e-8) -> np.ndarray:
    """Аппроксимация градиента барьерной функции конечными разностями."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n):
        x_plus = x.copy()
        x_minus = x.copy()
        x_plus[i] += eps
        x_minus[i] -= eps

        grad[i] = (barrier_function(x_plus, constraints, r, barrier_type) -
                   barrier_function(x_minus, constraints, r, barrier_type)) / (2 * eps)

    return grad

```

Метод модифицированных функций Лагранжа

(lagrange_methods.py)

```

"""
Модуль с методом модифицированных функций Лагранжа
"""

import numpy as np
from typing import Callable, List, Dict, Optional

def modified_lagrange_method(f: Callable, grad_f: Callable, x0: np.ndarray,
                             constraints: List[Callable],
                             lambda0: Optional[np.ndarray] = None,
                             mu0: Optional[np.ndarray] = None,
                             r0: float = 1.0, C: float = 2.0,
                             eps: float = 1e-6, max_outer_iter: int = 50,
                             max_inner_iter: int = 100) -> Dict:
    """
    Метод модифицированных функций Лагранжа (метод множителей Лагранжа).

    Args:
        f: Целевая функция
        grad_f: Градиент целевой функции
        x0: Начальная точка
        constraints: Список функций ограничений  $g_j(x) \leq 0$ 
        lambda0: Начальные множители для ограничений-равенств (не используется)
        mu0: Начальные множители для ограничений-неравенств
        r0: Начальный параметр штрафа
    """

```

C: Коэффициент увеличения штрафа
eps: Точность останова
max_outer_iter: Макс. число внешних итераций
max_inner_iter: Макс. число итераций внутренней оптимизации

Returns:

Словарь с результатами оптимизации
"""

m = len(constraints) # Число ограничений-неравенств

Инициализация множителей Лагранжа

if mu0 is None:

mu = np.zeros(m)

else:

mu = mu0.copy()

history = {

'x': [x0.copy()],

'f': [f(x0)],

'r': [r0],

'mu': [mu.copy()],

'feasible': [all(g(x0) <= 1e-6 for g in constraints)]

}

x = x0.copy()

r = r0

for k in range(max_outer_iter):

Модифицированная функция Лагранжа

def L(x):

val = f(x)

for j, g in enumerate(constraints):

g_val = g(x)

Для ограничений-неравенств

combined = mu[j] + r * g_val

val += (1 / (2 * r)) * (max(0, combined)**2 - mu[j]**2)

return val

def grad_L(x):

grad = grad_f(x)

for j, g in enumerate(constraints):

g_val = g(x)

combined = mu[j] + r * g_val

if combined > 0:

Градиент ограничения (аппроксимация)

grad_g = constraint_gradient_approx(x, constraints[j])

grad += combined * grad_g

return grad

Минимизация модифицированной функции Лагранжа

try:

from optimization import lbfgs

result = lbfgs(L, grad_L, x, max_iter=max_inner_iter)

except ImportError:

from scipy.optimize import minimize

result = minimize(L, x, method='L-BFGS-B', jac=grad_L)

result = {'x_star': result.x, 'f_star': result.fun, 'iterations': result.nit}

```

x_new = result['x_star']

# Обновление множителей Лагранжа
mu_new = np.zeros(m)
for j, g in enumerate(constraints):
    g_val = g(x_new)
    mu_new[j] = max(0, mu[j] + r * g_val)

# Проверка окончания
max_violation = max(abs(g(x_new)) for g in constraints)
feasible, _ = check_constraints_wrapper(x_new, constraints)

history['x'].append(x_new.copy())
history['f'].append(f(x_new))
history['r'].append(r)
history['mu'].append(mu_new.copy())
history['feasible'].append(feasible)

# Критерий останова
mu_change = np.linalg.norm(mu_new - mu)
if max_violation <= eps and mu_change <= eps:
    return {
        'x_star': x_new,
        'f_star': f(x_new),
        'iterations': k + 1,
        'lagrange_multipliers': mu_new,
        'feasible': feasible,
        'history': history,
        'method': 'modified_lagrange'
    }

# Обновление параметров
mu = mu_new
if max_violation > eps / 10:
    r = r * C # Увеличиваем штраф если нарушения велики
x = x_new

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': max_outer_iter,
    'lagrange_multipliers': mu,
    'feasible': check_constraints_wrapper(x, constraints)[0],
    'history': history,
    'method': 'modified_lagrange',
    'converged': False
}

def constraint_gradient_approx(x: np.ndarray, g: Callable,
                             eps: float = 1e-8) -> np.ndarray:
    """Аппроксимация градиента ограничения конечными разностями."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n):
        x_plus = x.copy()

```

```

    x_minus = x.copy()
    x_plus[i] += eps
    x_minus[i] -= eps

    grad[i] = (g(x_plus) - g(x_minus)) / (2 * eps)

return grad

def check_constraints_wrapper(x: np.ndarray, constraints: List[Callable],
                             tol: float = 1e-6) -> tuple:
    """Проверка выполнения ограничений."""
    violations = []
    for i, g in enumerate(constraints):
        g_val = g(x)
        if g_val > tol:
            violations.append((i, g_val))

    return len(violations) == 0, {
        'violations': violations,
        'max_violation': max([v[1] for v in violations]) if violations else 0.0,
        'feasible': len(violations) == 0
    }

```

Метод проекции градиента (projection_methods.py)

```

"""
Модуль с методом проекции градиента
"""

import numpy as np
from typing import Callable, List, Dict, Optional, Tuple

def projected_gradient_method(f: Callable, grad_f: Callable, x0: np.ndarray,
                              constraints: List[Callable],
                              grad_constraints: Optional[List[Callable]] = None,
                              eps1: float = -1e-6, eps2: float = 1e-8,
                              max_iter: int = 500,
                              line_search: str = 'armijo') -> Dict:
    """
    Метод проекции градиента для задач с ограничениями-неравенствами.

    Согласно методичке:
    - eps1 <= 0 для определения активных ограничений (eps1 <= g_j(x) <= 0)
    - eps2 > 0 для проверки сходимости

    Args:
        f: Целевая функция
        grad_f: Градиент целевой функции
        x0: Начальная точка
        constraints: Список функций ограничений  $g_j(x) \leq 0$ 
        grad_constraints: Градиенты ограничений (опционально)
        eps1: Точность определения активных ограничений (eps1 <= 0)
        eps2: Точность по направлению (eps2 > 0)
        max_iter: Максимальное число итераций
        line_search: Тип линейного поиска ('armijo', 'exact')

    Returns:

```

```

Словарь с результатами оптимизации
"""
n = len(x0)
m = len(constraints)

history = {
    'x': [x0.copy()],
    'f': [f(x0)],
    'step': [0.0],
    'active': [[]],
    'feasible': [check_constraints_simple(x0, constraints)]
}

x = x0.copy()

# Проверяем допустимость начальной точки
feasible, info = check_constraints_simple(x, constraints, return_info=True)
if not feasible:
    print(f"Начальная точка недопустима! Нарушения: {info['violations']}")
    x = find_feasible_point_simple(x, constraints)

for k in range(max_iter):
    # Определяем активные ограничения
    active_set = get_active_constraints(x, constraints, eps=1e-4)

    # Вычисляем проекцию антиградиента
    if len(active_set) == 0:
        # Нет активных ограничений - просто антиградиент
        d = -grad_f(x)
    else:
        # Есть активные ограничения - вычисляем проекцию
        d = compute_search_direction(x, grad_f, constraints,
                                     grad_constraints, active_set)

    # Проверка критерия остановки
    if np.linalg.norm(d) < eps1:
        # Проверяем условия Куна-Таккера
        lambdas = compute_lagrange_multipliers(x, grad_f, constraints,
                                                grad_constraints, active_set)
        if all(l >= -eps1 for l in lambdas):
            return {
                'x_star': x,
                'f_star': f(x),
                'iterations': k,
                'active_constraints': active_set,
                'lagrange_multipliers': lambdas,
                'feasible': check_constraints_simple(x, constraints),
                'history': history,
                'method': 'projected_gradient'
            }

    # Линейный поиск
    if line_search == 'armijo':
        alpha = armijo_line_search(f, x, d, grad_f, constraints)
    else:
        alpha = exact_line_search(f, x, d, constraints)

```

```

# Обновление точки
x_new = x + alpha * d

# Проверка критериев останковки
if (np.linalg.norm(x_new - x) < eps1 and
    abs(f(x_new) - f(x)) < eps2):
    return {
        'x_star': x_new,
        'f_star': f(x_new),
        'iterations': k + 1,
        'active_constraints': get_active_constraints(x_new, constraints),
        'feasible': check_constraints_simple(x_new, constraints),
        'history': history,
        'method': 'projected_gradient'
    }

# Запись в историю
history['x'].append(x_new.copy())
history['f'].append(f(x_new))
history['step'].append(alpha)
history['active'].append(get_active_constraints(x_new, constraints))
history['feasible'].append(check_constraints_simple(x_new, constraints))

x = x_new

return {
    'x_star': x,
    'f_star': f(x),
    'iterations': max_iter,
    'active_constraints': get_active_constraints(x, constraints),
    'feasible': check_constraints_simple(x, constraints),
    'history': history,
    'method': 'projected_gradient',
    'converged': False
}

def get_active_constraints(x: np.ndarray, constraints: List[Callable],
                          eps: float = 1e-4) -> List[int]:
    """Определить индексы активных ограничений."""
    active = []
    for i, g in enumerate(constraints):
        if abs(g(x)) < eps:
            active.append(i)
    return active

def compute_search_direction(x: np.ndarray, grad_f: Callable,
                             constraints: List[Callable],
                             grad_constraints: Optional[List[Callable]],
                             active_set: List[int]) -> np.ndarray:
    """
    Вычислить направление поиска как проекцию антиградиента.


$$d = -[E - A^T(AA^T)^{-1}A]\nabla f(x)$$

    где A - матрица градиентов активных ограничений
    """
    n = len(x)
    grad = grad_f(x)

```

```

if len(active_set) == 0:
    return -grad

# Вычисляем градиенты активных ограничений
if grad_constraints is not None:
    A = np.zeros((len(active_set), n))
    for i, j in enumerate(active_set):
        A[i, :] = grad_constraints[j](x)
else:
    # Аппроксимация градиентов
    A = np.zeros((len(active_set), n))
    for i, j in enumerate(active_set):
        A[i, :] = constraint_gradient(x, constraints[j])

# Проекция:  $d = -[E - A^T(AA^T)^{-1}A]\nabla f$ 
try:
    ATA_inv = np.linalg.inv(A @ A.T)
    P = np.eye(n) - A.T @ ATA_inv @ A
    d = -P @ grad
except np.linalg.LinAlgError:
    # Если матрица вырождена, используем псевдообратную
    try:
        ATA_pinv = np.linalg.pinv(A @ A.T)
        P = np.eye(n) - A.T @ ATA_pinv @ A
        d = -P @ grad
    except:
        d = -grad

return d

def compute_lagrange_multipliers(x: np.ndarray, grad_f: Callable,
                                constraints: List[Callable],
                                grad_constraints: Optional[List[Callable]],
                                active_set: List[int]) -> np.ndarray:
    """Вычислить множители Лагранжа для активных ограничений."""
    m = len(constraints)
    lambdas = np.zeros(m)

    if len(active_set) == 0:
        return lambdas

    n = len(x)
    grad = grad_f(x)

    # Вычисляем градиенты активных ограничений
    if grad_constraints is not None:
        A = np.zeros((len(active_set), n))
        for i, j in enumerate(active_set):
            A[i, :] = grad_constraints[j](x)
    else:
        A = np.zeros((len(active_set), n))
        for i, j in enumerate(active_set):
            A[i, :] = constraint_gradient(x, constraints[j])

    #  $\lambda = -(AA^T)^{-1}A^T \nabla f$ 
    try:

```



```

ATA_inv = np.linalg.inv(A @ A.T)
lambdas_active = -ATA_inv @ A @ grad

for i, j in enumerate(active_set):
    lambdas[j] = lambdas_active[i]
except:
    pass

return lambdas

def armijo_line_search(f: Callable, x: np.ndarray, d: np.ndarray,
                      grad_f: Callable, constraints: List[Callable],
                      alpha0: float = 1.0, rho: float = 0.5,
                      c: float = 1e-4) -> float:
    """Линейный поиск Армихо с учётом ограничений."""
    alpha = alpha0
    f_x = f(x)
    grad_dot_d = np.dot(grad_f(x), d)

    for _ in range(50):
        x_new = x + alpha * d

        # Проверка допустимости
        feasible = check_constraints_simple(x_new, constraints)

        # Условие Армихо
        if feasible and f(x_new) <= f_x + c * alpha * grad_dot_d:
            return alpha

        alpha *= rho

    return alpha

def exact_line_search(f: Callable, x: np.ndarray, d: np.ndarray,
                     constraints: List[Callable],
                     alpha_max: float = 10.0,
                     n_points: int = 100) -> float:
    """Точный линейный поиск с ограничениями."""
    # Находим максимальный допустимый шаг
    alpha_feasible = alpha_max

    for g in constraints:
        # Решаем квадратное уравнение  $g(x + \alpha d) = 0$ 
        # Для простоты используем бисекцию
        alpha_low, alpha_high = 0.0, alpha_max
        for _ in range(50):
            alpha_mid = (alpha_low + alpha_high) / 2
            if g(x + alpha_mid * d) > 0:
                alpha_high = alpha_mid
            else:
                alpha_low = alpha_mid
        alpha_feasible = min(alpha_feasible, alpha_high)

    # Минимизация на интервале [0, alpha_feasible]
    from scipy.optimize import minimize_scalar

    def phi(alpha):

```

```

    return f(x + alpha * d)

result = minimize_scalar(phi, bounds=(0, alpha_feasible), method='bounded')
return result.x

def constraint_gradient(x: np.ndarray, g: Callable, eps: float = 1e-8) -> np.ndarray:
    """Аппроксимация градиента ограничения."""
    n = len(x)
    grad = np.zeros(n)

    for i in range(n):
        x_plus = x.copy()
        x_minus = x.copy()
        x_plus[i] += eps
        x_minus[i] -= eps

        grad[i] = (g(x_plus) - g(x_minus)) / (2 * eps)

    return grad

def check_constraints_simple(x: np.ndarray, constraints: List[Callable],
                            tol: float = 1e-6,
                            return_info: bool = False) -> bool:
    """Проверка выполнения ограничений."""
    violations = []
    for i, g in enumerate(constraints):
        g_val = g(x)
        if g_val > tol:
            violations.append((i, g_val))

    if return_info:
        return len(violations) == 0, {
            'violations': violations,
            'max_violation': max([v[1] for v in violations]) if violations else 0.0
        }
    return len(violations) == 0

def find_feasible_point_simple(x0: np.ndarray, constraints: List[Callable],
                               max_iter: int = 100) -> np.ndarray:
    """Найти допустимую точку."""
    x = x0.copy()
    n = len(x)

    # Для ограничений неотрицательности (g2, g3)
    for i in range(n):
        if x[i] < 0:
            x[i] = 0.1 # Небольшое положительное значение

    # Для g1:  $x_1^2 - x_2^2 - 3 \leq 0 \Rightarrow x_1^2 \leq x_2^2 + 3$ 
    if len(constraints) >= 1:
        g1_val = x[0]**2 - x[1]**2 - 3
        if g1_val > 0:
            x[0] = np.sqrt(x[1]**2 + 3) + 0.1 # Увеличиваем x1 чтобы удовлетворить g1

    return x

```

Класс оптимизатора (optimizer.py)

```
"""
Класс ConstrainedOptimizer для удобного запуска оптимизации с ограничениями
"""

import numpy as np
import time
from typing import Callable, List, Dict, Optional, Tuple

from .functions import (
    rosenbrock,
    rosenbrock_gradient,
    create_rosenbrock_function,
    get_constraints_variant3, # Обновлено для третьего варианта
    check_constraints
)

from .penalty_methods import (
    penalty_method,
    barrier_method,
    combined_penalty_barrier_method
)

from .lagrange_methods import (
    modified_lagrange_method
)

from .projection_methods import (
    projected_gradient_method
)

class ConstrainedOptimizer:
    """
    Класс для выполнения оптимизации с ограничениями.
    """

    def __init__(self, f: Callable, grad_f: Callable, x0: np.ndarray,
                  constraints: List[Callable],
                  eps: float = 1e-6, max_iter: int = 100):
        """
        Инициализация оптимизатора.

        Args:
            f: Целевая функция
            grad_f: Градиент целевой функции
            x0: Начальная точка
            constraints: Список функций ограничений  $g_j(x) \leq 0$ 
            eps: Точность останова
            max_iter: Максимальное число итераций
        """
        self.f = f
        self.grad_f = grad_f
        self.x0 = x0
        self.constraints = constraints
        self.eps = eps
        self.max_iter = max_iter
```

```

self.results: Dict[str, Dict] = {}

def run_all_methods(self, verbose: bool = True) -> Dict[str, Dict]:
    """
    Запустить все методы оптимизации с ограничениями.
    """
    methods = [
        ('Метод штрафных функций', self._run_penalty),
        ('Метод барьерных функций', self._run_barrier),
        ('Комбинированный метод', self._run_combined),
        ('Метод множителей Лагранжа', self._run_lagrange),
        ('Метод проекции градиента', self._run_projection)
    ]

    if verbose:
        print("\n" + "=" * 70)
        print("ЗАПУСК ВСЕХ МЕТОДОВ УСЛОВНОЙ ОПТИМИЗАЦИИ")
        print("=" * 70)

    for name, method in methods:
        if verbose:
            print(f"\nЗапуск метода: {name}...")

        start_time = time.time()
        result = method()
        elapsed_time = time.time() - start_time

        self.results[name] = {
            **result,
            'time': elapsed_time
        }

        if verbose:
            print(f" Найдено: x* = {result['x_star']}")
            print(f" f(x*) = {result['f_star']:.8f}")
            print(f" Итераций: {result['iterations']}")
            print(f" Время: {elapsed_time:.4f} сек")
            print(f" Допустимо: {result['feasible']}")

    return self.results

def _run_penalty(self) -> Dict:
    """Запустить метод штрафных функций."""
    return penalty_method(
        self.f, self.grad_f, self.x0, self.constraints,
        r0=1.0, C=10.0, eps=self.eps, max_outer_iter=self.max_iter
    )

def _run_barrier(self) -> Dict:
    """Запустить метод барьерных функций."""
    return barrier_method(
        self.f, self.grad_f, self.x0, self.constraints,
        r0=1.0, C=10.0, eps=self.eps, max_outer_iter=self.max_iter
    )

def _run_combined(self) -> Dict:
    """Запустить комбинированный метод."""

```

```

return combined_penalty_barrier_method(
    self.f, self.grad_f, self.x0, self.constraints,
    r0=1.0, C=10.0, eps=self.eps, max_outer_iter=self.max_iter,
    alpha=0.5
)

def _run_lagrange(self) -> Dict:
    """Запустить метод модифицированных функций Лагранжа."""
    return modified_lagrange_method(
        self.f, self.grad_f, self.x0, self.constraints,
        r0=1.0, C=2.0, eps=self.eps, max_outer_iter=self.max_iter
    )

def _run_projection(self) -> Dict:
    """Запустить метод проекции градиента."""
    return projected_gradient_method(
        self.f, self.grad_f, self.x0, self.constraints,
        eps1=self.eps, max_iter=self.max_iter * 10
    )

def get_best_method(self, criterion: str = 'f_star') -> Tuple[str, Dict]:
    """
    Получить лучший метод по заданному критерию.

    Args:
        criterion: 'f_star', 'iterations', 'time', 'accuracy'
    """
    if not self.results:
        raise ValueError("Сначала запустите методы оптимизации")

    if criterion == 'f_star':
        best_name = min(self.results.keys(),
                        key=lambda k: self.results[k]['f_star'])
    elif criterion == 'iterations':
        best_name = min(self.results.keys(),
                        key=lambda k: self.results[k]['iterations'])
    elif criterion == 'time':
        best_name = min(self.results.keys(),
                        key=lambda k: self.results[k]['time'])
    elif criterion == 'accuracy':
        def accuracy(name):
            result = self.results[name]
            penalty = sum(max(0, g(result['x_star']))**2
                          for g in self.constraints)
            return penalty
        best_name = min(self.results.keys(), key=accuracy)
    else:
        raise ValueError(f"Неизвестный критерий: {criterion}")

    return best_name, self.results[best_name]

def print_comparison_table(self):
    """Вывести сравнительную таблицу результатов."""
    if not self.results:
        print("Нет результатов для сравнения")
    return

```

```

print("\n" + "=" * 90)
print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА МЕТОДОВ УСЛОВНОЙ ОПТИМИЗАЦИИ")
print("=" * 90)
print(f'{"Метод":<35} {"Итер.":<8} {"f(x*)":<15} '
      f'{"Время (с)":<10} {"Допустимо":<10}')
print("-" * 90)

for name, result in self.results.items():
    feasible_str = "✓" if result['feasible'] else "X"
    print(f'{"name":<35} {"result[\"iterations\"]":<8} '
          f'{"result[\"f_star\"]":<15.8f} {"result[\"time\"]":<10.4f} '
          f'{"feasible_str":<10}')

print("=" * 90)

# Проверка допустимости решений
print("\nПРОВЕРКА ДОПУСТИМОСТИ РЕШЕНИЙ:")
print("-" * 90)

for name, result in self.results.items():
    x = result['x_star']
    feasible, info = check_constraints(x, self.constraints)

    print(f'\n{name}:')
    print(f'  x* = {x}')
    print(f'  f(x*) = {result["f_star"]:.8f}')
    print(f'  Допустимо: {feasible}')

    if info['violations']:
        print(f'  Нарушения:')
        for i, val in info['violations']:
            print(f'    g{i+1}(x) = {val:.6f} > 0')
    else:
        print(f'  Все ограничения выполнены')
        print(f'  Значения ограничений:')
        for i, g in enumerate(self.constraints):
            print(f'    g{i+1}(x) = {g(x):.6f} ≤ 0')

#
=====
=
# Вспомогательная функция для создания оптимизатора для варианта 3
#
=====
=

def create_optimizer_variant3(x0: Optional[np.ndarray] = None,
                              eps: float = 1e-6,
                              max_iter: int = 100) -> ConstrainedOptimizer:
    """
    Создать оптимизатор для варианта 3.

    Args:
        x0: Начальная точка (по умолчанию [1.5, 1.5])
        eps: Точность
        max_iter: Макс. итераций

```

Returns:

```
ConstrainedOptimizer для варианта 3
"""
if x0 is None:
    x0 = np.array([1.5, 1.5]) # Новая начальная точка для третьего варианта

f, grad_f, _ = create_rosenbrock_function(a=80, b=3, f0=110) # Обновленные параметры
constraints = get_constraints_variant3() # Обновлено для третьего варианта

return ConstrainedOptimizer(f, grad_f, x0, constraints, eps, max_iter)
```

Класс визуализации (visualization.py)

```
"""
Модуль визуализации для методов условной оптимизации
"""

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
from typing import Dict, List, Callable, Optional, Tuple
import os

def plot_feasible_region(constraints: List[Callable],
                        x_bounds: Tuple = (-4.5, 1.5),
                        save_path: Optional[str] = None):
    """
    Построить область допустимых решений для n=2.
    """
    x1 = np.linspace(x_bounds[0], x_bounds[1], 200)
    x2 = np.linspace(x_bounds[0], x_bounds[1], 200)
    X1, X2 = np.meshgrid(x1, x2)

    # ИСПРАВЛЕНО: Изменяем на 2x2 сетку
    fig, axes = plt.subplots(2, 2, figsize=(10, 10))
    axes = axes.flatten() # Преобразуем в одномерный массив для удобной итерации

    for i, (g, ax) in enumerate(zip(constraints, axes)):
        # Для 3D переменных фиксируем x3
        Z = np.zeros_like(X1)
        for j in range(len(x1)):
            for k in range(len(x2)):
                # Для варианта 3:
                Z[k, j] = g(np.array([x1[j], x2[k]])) # Для n=2, без фиксации x3

        # Область где g(x) ≤ 0
        feasible = Z <= 0

        ax.contourf(X1, X2, feasible, levels=[0.5, 1], colors=['lightgreen'], alpha=0.5)
        ax.contour(X1, X2, Z, levels=[0], colors='red', linewidths=2)
        ax.contourf(X1, X2, Z, levels=[-100, 0], colors=['lightgreen'], alpha=0.3)

        ax.set_xlabel('x1')
        ax.set_ylabel('x2')
        ax.set_title(f'g{i+1}(x) ≤ 0')
        ax.grid(True, alpha=0.3)
```

```

fig.suptitle('Область допустимых решений (индивидуальные ограничения)', fontsize=16,
fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'Область допустимых решений сохранена в '{save_path}''')

plt.show()

def plot_all_constraints_grid(constraints: List[Callable],
                             save_path: Optional[str] = None):
    """
    Построить сетку 2x2 с визуализацией каждого ограничения.
    """
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.flatten()

    constraint_names = [
        'g1:  $x_1^2 - x_2^2 - 3 \leq 0$ ',
        'g2:  $-x_1 \leq 0$  ( $x_1 \geq 0$ )',
        'g3:  $-x_2 \leq 0$  ( $x_2 \geq 0$ )'
    ]

    x1 = np.linspace(-0.5, 1.5, 200)
    x2 = np.linspace(-0.5, 1.5, 200)
    X1, X2 = np.meshgrid(x1, x2)
    x3_fixed = 1.0

    for i, (g, name, ax) in enumerate(zip(constraints, constraint_names, axes)):
        # Вычисляем значение ограничения
        Z = np.zeros_like(X1)
        for j in range(len(x1)):
            for k in range(len(x2)):
                Z[k, j] = g(np.array([x1[j], x2[k]]))

        # Область где  $g(x) \leq 0$ 
        feasible = Z <= 0

        ax.contourf(X1, X2, feasible, levels=[0.5, 1], colors=['lightgreen'], alpha=0.5)
        ax.contour(X1, X2, Z, levels=[0], colors='red', linewidths=2)

        # Добавим цветовую шкалу
        contour_full = ax.contourf(X1, X2, Z, levels=30, cmap='RdYlGn_r', alpha=0.6)
        cbar = fig.colorbar(contour_full, ax=ax)
        cbar.set_label('g(x)')

        ax.set_xlabel('x1')
        ax.set_ylabel('x2')
        ax.set_title(f'{name}\n(проекция при  $x_3 = \{x_3\_fixed\}$ )')
        ax.grid(True, alpha=0.3)
        ax.set_aspect('equal')
        ax.set_xlim(-0.5, 1.5)
        ax.set_ylim(-0.5, 1.5)

    fig.suptitle('Визуализация каждого ограничения', fontsize=16, fontweight='bold')

```



```

plt.tight_layout(rect=[0, 0.03, 1, 0.95])

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'Сетка ограничений сохранена в '{save_path}''')

plt.show()

def plot_trajectories_3d(results: Dict, constraints: List[Callable],
                        x_bounds: Tuple = (0, 1.5),
                        save_path: Optional[str] = None):
    """
    Построить траектории методов в 3D.
    """
    fig = plt.figure(figsize=(14, 12))
    ax = fig.add_subplot(111, projection='3d')

    # === 1. Визуализация границ ограничений ===

    # Сетка для поверхностей
    x_range = np.linspace(x_bounds[0], x_bounds[1], 20)
    y_range = np.linspace(x_bounds[0], x_bounds[1], 20)

    # Поверхность  $g_1(x) = 0 \Rightarrow x_3 = x_1^2 + x_2^2 - 1$ 
    X1_g1, X2_g1 = np.meshgrid(x_range, y_range)
    X3_g1 = X1_g1**2 + X2_g1**2 - 1
    ax.plot_surface(X1_g1, X2_g1, X3_g1, alpha=0.5, cmap=cm.viridis)

    # Определяем z_bounds на основе поверхности g1
    z_bounds = (np.min(X3_g1), np.max(X1_g1**2 + X2_g1**2))
    z_range = np.linspace(z_bounds[0], z_bounds[1], 20)

    # Плоскость  $g_2(x) = 0 \Rightarrow x_1 = 0$  (плоскость Y-Z)
    Y2_g2, Z2_g2 = np.meshgrid(y_range, z_range)
    X2_g2 = np.zeros_like(Y2_g2)
    ax.plot_surface(X2_g2, Y2_g2, Z2_g2, alpha=0.3, color='red')

    # Плоскость  $g_3(x) = 0 \Rightarrow x_2 = 0$  (плоскость X-Z)
    X3_g3, Z3_g3 = np.meshgrid(x_range, z_range)
    Y3_g3 = np.zeros_like(X3_g3)
    ax.plot_surface(X3_g3, Y3_g3, Z3_g3, alpha=0.3, color='green')

    # === 2. Траектории методов ===
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(results))))

    for (name, result), color in zip(results.items(), colors):
        if 'history' in result and 'x' in result['history']:
            x_path = np.array(result['history']['x'])
            ax.plot(x_path[:, 0], x_path[:, 1], x_path[:, 2],
                    'o-', color=color, label=name, markersize=3, linewidth=1.5, alpha=0.8)

            # Начало и конец
            ax.scatter(x_path[0, 0], x_path[0, 1], x_path[0, 2],
                       color='black', s=80, marker='s', alpha=0.9, label='Ctrap' if name == list(results.keys())[0]
                       else "")
            ax.scatter(x_path[-1, 0], x_path[-1, 1], x_path[-1, 2],
                       color=color, s=120, marker='*', alpha=0.9)

```

```

# Глобальный минимум (1,1,1)
ax.scatter(1, 1, color='magenta', s=250, marker='*', label='Минимум (1,1)', edgecolors='black')

ax.set_xlabel('x1', fontweight='bold')
ax.set_ylabel('x2', fontweight='bold')
ax.set_zlabel('x3', fontweight='bold')
ax.set_title('Траектории методов условной оптимизации в 3D', fontsize=16)

# Собираем легенду, избегая дубликатов
handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax.legend(by_label.values(), by_label.keys(), loc='center left', bbox_to_anchor=(-0.2, 0.5), fontsize=8)

ax.view_init(elev=20, azim=-60) # Устанавливаем хороший ракурс

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'3D траектории сохранены в '{save_path}'')

plt.show()

def plot_convergence(results: Dict, f_opt: float = 100.0,
                    save_path: Optional[str] = None):
    """
    Построить графики сходимости методов с нормализацией по итерациям.
    """
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))

    method_names = list(results.keys())
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(method_names))))

    # Собираем все данные для нормализации
    all_f = []
    all_violations = []
    max_iter = 0
    for name, result in results.items():
        if 'history' in result and 'f' in result['history']:
            all_f.extend(result['history']['f'])
            max_iter = max(max_iter, len(result['history']['f']))
        if 'history' in result and 'x' in result['history']:
            for x in result['history']['x']:
                g1 = x[0]**2 - x[1]**2 - 3
                g2 = -x[0]
                g3 = -x[1]
                all_violations.append(max(0, g1, g2, g3))

    f_min, f_max = min(all_f), max(all_f)
    f_range = f_max - f_min if f_max > f_min else 1
    violation_max = max(all_violations) if all_violations else 1

    # 1. Сходимость по функции (НОРМИРОВАННАЯ по X и Y)
    ax = axes[0, 0]
    for (name, result), color in zip(results.items(), colors):
        if 'history' in result and 'f' in result['history']:

```

```

f_history = np.array(result['history']['f'])
f_norm = (f_history - f_min) / f_range # Нормализация Y к [0, 1]

# Нормализация X (итерации) к [0, 1]
n_iter = len(f_history)
x_norm = np.linspace(0, 1, n_iter)

ax.plot(x_norm, f_norm, label=name, linewidth=2, color=color)

ax.axhline(y=0, color='red', linestyle='--', label=f'Оптимум (норм.)')
ax.set_xlabel('Итерации (норм.)')
ax.set_ylabel('f(x) (норм.)')
ax.set_title('Сходимость по значению функции (нормализованная)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# 2. Нарушение ограничений (НОРМИРОВАННОЕ по X и Y)
ax = axes[0, 1]
for (name, result), color in zip(results.items(), colors):
    if 'history' in result and 'x' in result['history']:
        violations = []
        for x in result['history']['x']:
            g1 = x[0]**2 - x[1]**2 - 3
            g2 = -x[0]
            g3 = -x[1]
            max_violation = max(0, g1, g2, g3)
            violations.append(max_violation / violation_max if violation_max > 0 else 0)

        # Нормализация X (итерации) к [0, 1]
        n_iter = len(violations)
        x_norm = np.linspace(0, 1, n_iter)

        ax.plot(x_norm, violations, label=name, linewidth=2, color=color)

ax.set_xlabel('Итерации (норм.)')
ax.set_ylabel('max violation (норм.)')
ax.set_title('Нарушение ограничений (нормализованное)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)

# 3. Параметр штрафа r
ax = axes[1, 0]
has_r = False
for (name, result), color in zip(results.items(), colors):
    if 'history' in result and 'r' in result['history']:
        has_r = True
        r_history = result['history']['r']
        # Нормализация X
        n_iter = len(r_history)
        x_norm = np.linspace(0, 1, n_iter)
        ax.semilogy(x_norm, r_history, label=name, linewidth=2, color=color)

if has_r:
    ax.set_xlabel('Итерации (норм.)')
    ax.set_ylabel('r (лог)')
    ax.set_title('Параметр штрафа')
    ax.legend(fontsize=8)

```

```

    ax.grid(True, alpha=0.3)
else:
    ax.text(0.5, 0.5, 'Параметр  $r$  не используется', ha='center', va='center')

# 4. Время vs Точность
ax = axes[1, 1]
times = [results[name]['time'] for name in method_names]
f_stars = [results[name]['f_star'] for name in method_names]

bars = ax.bar(method_names, times, color=colors, alpha=0.8)
ax.set_ylabel('Время (сек)')
ax.set_title('Время выполнения')
ax.tick_params(axis='x', rotation=45)

for bar, val, f_val in zip(bars, times, f_stars):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.001,
            f'{val:.3f} c\nf={f_val:.2f}', ha='center', va='bottom', fontsize=8)

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'Графики сходимости сохранены в '{save_path}'')

plt.show()

def plot_constraints_check(results: Dict, constraints: List[Callable],
                          save_path: Optional[str] = None):
    """
    Построить проверку ограничений для лучших решений.
    """
    fig, ax = plt.subplots(figsize=(12, 6))

    method_names = list(results.keys())
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(method_names))))

    n_constraints = len(constraints)
    x = np.arange(n_constraints)
    width = 0.8 / len(method_names)

    for i, (name, result) in enumerate(results.items()):
        x_star = result['x_star']
        g_values = [g(x_star) for g in constraints]

        bars = ax.bar(x + i*width - 0.4, g_values, width, label=name, color=colors[i], alpha=0.8)

        # Добавляем значения над столбцами
        for bar, g_val in zip(bars, g_values):
            height = bar.get_height()
            # Позиция текста зависит от знака значения
            y_pos = height + (0.02 if height >= 0 else -0.03)
            va = 'bottom' if height >= 0 else 'top'

            ax.text(bar.get_x() + bar.get_width()/2, y_pos,
                    f'{g_val:.4f}', ha='center', va=va, fontsize=7, rotation=45)

    ax.axhline(y=0, color='red', linestyle='-', linewidth=2, label='g(x)=0')

```

```

ax.set_xlabel('Ограничение')
ax.set_ylabel('g(x)')
ax.set_title('Проверка ограничений для найденных решений\n(значения указаны над столбцами)')
ax.set_xticks(range(n_constraints))
ax.set_xticklabels([f'g {i+1}' for i in range(n_constraints)])
ax.legend(fontsize=8, loc='upper right')
ax.grid(True, alpha=0.3)

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'Проверка ограничений сохранена в '{save_path}''')

plt.show()

def plot_trajectories_2d_combined(results: Dict, constraints: List[Callable],
                                  save_path: Optional[str] = None):
    """
    Построить все траектории методов на одном 2D графике (проекция x1-x2).
    """
    fig, ax = plt.subplots(figsize=(10, 8))

    # Область допустимых решений
    x1 = np.linspace(0, 1.5, 200)
    x2 = np.linspace(0, 1.5, 200)
    X1, X2 = np.meshgrid(x1, x2)

    # Для 2D проекции фиксируем x3 = 1 (значение в точке минимума)
    x3_fixed = 1.0

    # g1:  $x_1^2 - x_2^2 - 3 \leq 0$ 
    Z1 = X1**2 - X2**2 - 3

    # g2:  $-x_1 \leq 0 \Rightarrow x_1 \geq 0$ 
    # g3:  $-x_2 \leq 0 \Rightarrow x_2 \geq 0$ 
    feasible = (Z1 <= 0) & (X1 >= 0) & (X2 >= 0)

    ax.contourf(X1, X2, feasible, levels=[0.5, 1], colors=['lightgreen'], alpha=0.3)
    ax.contour(X1, X2, Z1, levels=[0], colors='blue', linewidths=2, linestyle='--', label='g1(x)=0')

    # Границы g2 и g3
    ax.axvline(x=0, color='red', linestyle='--', linewidth=2, label='g2(x)=0 (x1=0)')
    ax.axhline(y=0, color='green', linestyle='--', linewidth=2, label='g3(x)=0 (x2=0)')

    # Траектории методов
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(results))))

    for (name, result), color in zip(results.items(), colors):
        if 'history' in result and 'x' in result['history']:
            x_path = np.array(result['history']['x'])
            ax.plot(x_path[:, 0], x_path[:, 1], 'o-', color=color,
                    label=name, markersize=3, linewidth=1.5, alpha=0.7)

    # Начало и конец
    ax.plot(x_path[0, 0], x_path[0, 1], 's', color=color, markersize=10, alpha=0.8)
    ax.plot(x_path[-1, 0], x_path[-1, 1], '*', color=color, markersize=12, alpha=0.8)

```

```

# Глобальный минимум
ax.plot(1, 1, 'r*', markersize=15, label='Минимум (1,1)')

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Все траектории методов (проекция x1-x2, x3=1)')
ax.legend(fontsize=7, loc='upper right')
ax.grid(True, alpha=0.3)
ax.set_aspect('equal')
ax.set_xlim(-0.5, 2.5) # Расширяем диапазон для g1
ax.set_ylim(-0.5, 2.5)

plt.tight_layout()

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches='tight')
    print(f'2D траектории (общий) сохранены в '{save_path}'')

plt.show()

def plot_trajectories_2d_individual(results: Dict, constraints: List[Callable],
                                   save_path: Optional[str] = None):
    """
    Построить отдельные 2D графики для каждого метода (общие для всех ограничений).
    """
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(results))))

    # Создаём директорию для индивидуальных графиков
    if save_path:
        os.makedirs(save_path, exist_ok=True)

    for (name, result), color in zip(results.items(), colors):
        fig, axes = plt.subplots(1, 2, figsize=(12, 5))

        if 'history' not in result or 'x' not in result['history']:
            continue

        x_path = np.array(result['history']['x'])

        # Область допустимых решений (все ограничения)
        x1 = np.linspace(0, 1.5, 200)
        x2 = np.linspace(0, 1.5, 200)
        X1, X2 = np.meshgrid(x1, x2)
        x3_fixed = 1.0

        Z1 = X1**2 - X2**2 - 3
        feasible = (Z1 <= 0) & (X1 >= 0) & (X2 >= 0)

        # График 1: Траектория на области допустимых решений
        ax = axes[0]
        ax.contourf(X1, X2, feasible, levels=[0.5, 1], colors=['lightgreen'], alpha=0.3)
        ax.contour(X1, X2, Z1, levels=[0], colors='blue', linewidths=2, linestyle='--')
        ax.axvline(x=0, color='red', linestyle='--', linewidth=1.5)
        ax.axhline(y=0, color='green', linestyle='--', linewidth=1.5)

        ax.plot(x_path[:, 0], x_path[:, 1], 'o-', color=color, markersize=4, linewidth=2, alpha=0.8)

```

```

ax.plot(x_path[0, 0], x_path[0, 1], 'gs', markersize=12, label='Старт', alpha=0.8)
ax.plot(x_path[-1, 0], x_path[-1, 1], 'r*', markersize=15, label='Финиш', alpha=0.8)

ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title(f'{name}\nТраектория (все ограничения)')
ax.legend(fontsize=8, loc='upper right')
ax.grid(True, alpha=0.3)
ax.set_aspect('equal')
ax.set_xlim(0, 1.5)
ax.set_ylim(0, 1.5)

# График 2: Сходимость по функции
ax = axes[1]
f_history = result['history'][:f]
ax.plot(range(len(f_history)), f_history, color=color, linewidth=2)
ax.axhline(y=100, color='red', linestyle='--', label='Оптимум (100)')
ax.set_xlabel('Итерация')
ax.set_ylabel('f(x)')
ax.set_title(f'{name}\nСходимость по значению функции')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3)

plt.tight_layout()

if save_path:
    safe_name = name.replace(' ', '_').replace('(', '').replace(')', '').replace('-', '_')
    file_path = os.path.join(save_path, f'trajectory_2d_{safe_name}.png')
    plt.savefig(file_path, dpi=150, bbox_inches='tight')
    print(f'Траектория '{name}' сохранена в '{file_path}'')

plt.show()

def plot_trajectories_per_constraint(results: Dict, constraints: List[Callable],
                                     save_path: Optional[str] = None):
    """
    Построить 5 графиков (по одному на метод), каждый с 4 подграфиками ограничений.
    5 методов × 1 график (4 подграфика) = 5 графиков.
    """
    colors = list(cm.get_cmap('tab10')(np.linspace(0, 1, len(results))))
    constraint_names = [
        'g1:  $x_1^2 - x_2^2 - 3 \leq 0$ ',
        'g2:  $-x_1 \leq 0$  ( $x_1 \geq 0$ )',
        'g3:  $-x_2 \leq 0$  ( $x_2 \geq 0$ )'
    ]

    # Создаём директорию
    if save_path:
        os.makedirs(save_path, exist_ok=True)

    for (method_name, result), method_color in zip(results.items(), colors):
        if 'history' not in result or 'x' not in result['history']:
            continue

        x_path = np.array(result['history'][:, 'x'])
        f_history = result['history'][:, 'f']

```

```

# Создаём 4 подграфика - по одному для каждого ограничения
fig, axes = plt.subplots(2, 2, figsize=(14, 12))
axes = axes.flatten()

# ИСПРАВЛЕНО: Расширяем диапазон, чтобы contour работал на границах x=0, y=0
x1 = np.linspace(-0.5, 1.5, 200)
x2 = np.linspace(-0.5, 1.5, 200)
X1, X2 = np.meshgrid(x1, x2)
x3_fixed = 1.0

for i, (g, g_name) in enumerate(zip(constraints, constraint_names)):
    ax = axes[i]

    # Вычисляем значение ограничения
    Z = np.zeros_like(X1)
    for j in range(len(x1)):
        for k in range(len(x2)):
            Z[k, j] = g(np.array([x1[j], x2[k]]))

    # Градиентная цветовая шкала
    contour_full = ax.contourf(X1, X2, Z, levels=50, cmap='RdYlGn_r', alpha=0.7)
    cbar = plt.colorbar(contour_full, ax=ax)
    cbar.set_label('g(x)')

    # Чёрная линия границы g(x) = 0 (универсальный метод)
    ax.contour(X1, X2, Z, levels=[0], colors='black', linewidths=3)

    # Для g4, где нет линии, добавим поясняющий текст
    if i == 3:
        ax.text(0.5, 0.5, 'g4: x3 ≥ 0\n(выполнено при x3=1, нет линии g(x)=0)',
              ha='center', va='center', fontsize=12, fontweight='bold',
              bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.8))

    # Траектория метода
    ax.plot(x_path[:, 0], x_path[:, 1], 'o-', color=method_color,
           markersize=5, linewidth=2.5, alpha=0.9, label='Траектория')
    ax.plot(x_path[0, 0], x_path[0, 1], 'gs', markersize=15, label='Старт', alpha=0.9, zorder=10)
    ax.plot(x_path[-1, 0], x_path[-1, 1], 'r*', markersize=20, label='Финиш', alpha=0.9, zorder=10)

    # Точка минимума (1,1)
    ax.plot(1, 1, 'b+', markersize=25, linewidth=3, label='Минимум (1,1)', zorder=10)

    ax.set_xlabel('x1', fontsize=12)
    ax.set_ylabel('x2', fontsize=12)
    ax.set_title(f'{g_name}\n(проекция при x3={x3_fixed})', fontsize=12, fontweight='bold')
    ax.legend(fontsize=9, loc='upper right')
    ax.grid(True, alpha=0.3)
    ax.set_aspect('equal')
    # ИСПРАВЛЕНО: Обновляем пределы, чтобы соответствовать новому диапазону
    ax.set_xlim(-0.5, 1.5)
    ax.set_ylim(-0.5, 1.5)

# Общий заголовок для всех 4 ограничений
fig.suptitle(f'{method_name}\nТраектория на фоне каждого ограничения\n'
            f'Финиш: f(x*)={f_history[-1]:.6f}, итераций={len(f_history)}',
            fontsize=14, fontweight='bold', y=1.02)

```



```

plt.tight_layout()

if save_path:
    safe_method = method_name.replace(' ', '_').replace('(', '').replace(')', '').replace('-', '_')
    file_path = os.path.join(save_path, f'trajectory_all_g_{safe_method}.png')
    plt.savefig(file_path, dpi=150, bbox_inches='tight')
    print(f'Траектория '{method_name}' (все g) сохранена в '{file_path}''')

plt.show()

def create_visualization(results: Dict, constraints: List[Callable],
                        f_opt: float = 100.0,
                        output_dir: str = '.'):
    """
    Создать все визуализации.
    """
    plots_dir = os.path.join(output_dir, 'plots')
    os.makedirs(plots_dir, exist_ok=True)

    print("\n" + "=" * 60)
    print("ГЕНЕРАЦИЯ ВИЗУАЛИЗАЦИИ")
    print("=" * 60)

    # 1. Область допустимых решений (все ограничения вместе)
    plot_feasible_region(
        constraints,
        save_path=os.path.join(plots_dir, 'feasible_region_all.png')
    )

    # 2. Сетка с визуализацией каждого ограничения
    plot_all_constraints_grid(
        constraints,
        save_path=os.path.join(plots_dir, 'constraints_g_all.png')
    )

    # 4. 2D траектории - все методы в одном
    plot_trajectories_2d_combined(
        results,
        constraints,
        save_path=os.path.join(plots_dir, 'trajectories_2d_combined.png')
    )

    # 5. 2D траектории - отдельные для каждого метода (5 графиков)
    plot_trajectories_2d_individual(
        results,
        constraints,
        save_path=os.path.join(plots_dir, 'trajectories_2d_per_method')
    )

    # 6. Траектории методов с каждым ограничением (20 графиков: 5 методов × 4 ограничения)
    plot_trajectories_per_constraint(
        results,
        constraints,
        save_path=os.path.join(plots_dir, 'trajectories_per_constraint')
    )

```

```
# 7. Графики сходимости
plot_convergence(
    results,
    f_opt,
    save_path=os.path.join(plots_dir, 'convergence.png')
)

# 8. Проверка ограничений
plot_constraints_check(
    results,
    constraints,
    save_path=os.path.join(plots_dir, 'constraints_check.png')
)

print(f"\nВсе графики сохранены в директорию '{plots_dir}'")
```

Вывод

В ходе выполнения лабораторной работы изучили алгоритмы условной оптимизации, разрабатывали программы реализации алгоритмов условной оптимизации, находили оптимальные решения для задач с учетом ограничений.